```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P03.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S03_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001④
    ///      Boot            UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P03.Boot
{
    /// &lt;summary&gt;
```

```
/// Boot     T001
/// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY_MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //              JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P03.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
```

```
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                  TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
```

```
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P03.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S03_Main";
        public const string Battle = "S03_Battle";
    }
    /// &lt;summary&gt;S03          →S03_Battle M5      ——           &lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        public void StartBattle()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P03_scene_01", "P03_unit_01");
        }
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// &lt;summary&gt;
    /// S03_Battle       BattleSim+TutorialFlow+BattleView   Tick
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
    /// HUD                     6
    /// &lt;/summary&gt;
    public sealed class BattleLauncher : MonoBehaviour
    {
        BattleSim _sim;
        TutorialFlow _tutorial;
        BattleView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        int _lastWave = -1, _lastLives = -1, _lastCrystals = -1, _lastKills = -1;
        void Awake()
        {
```

```
    Application.targetFrameRate = 30;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    _sim = new BattleSim();
    _view = gameObject.AddComponent&lt;BattleView&gt;();
    _view.Initialize(canvas, _sim);
    _telemetry = new Sy.Telemetry.TelemetryService();
    _telemetry.AddSink(new TelemetryDebugSink());
    _tutorial = new TutorialFlow(
        _sim,
        hint =&gt;
        {
            Debug.Log("[HINT] " + hint);
            _view.SetHint(hint);
        },
        step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
            new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
    _view.SlotTapped += OnSlotTapped;
    _view.TianjiPicked += OnTianjiPicked;
    _view.QingLongRequested += OnQingLongRequested;
    _view.EnemyTapped += OnEnemyTapped;
    _sim.TowerPlaced += OnTowerPlaced;
    _sim.TianjiOffered += OnTianjiOffered;
    _sim.QingLongFired += OnQingLongFired;
    _sim.StateChanged += OnStateChanged;
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Wave != _lastWave) { _lastWave = _sim.Wave; _view.SetWave(_lastWave); }
    if (_sim.Lives != _lastLives) { _lastLives = _sim.Lives; _view.SetLives(_lastLives); }
    if (_sim.Crystals != _lastCrystals) { _lastCrystals = _sim.Crystals; _view.SetCrystals(_lastCrystals); }
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
}
void OnSlotTapped(int slot)
{
    _sim.PlaceTower(slot);
}
void OnTianjiPicked(int index)
{
    _sim.PickTianji(index);
}
void OnQingLongRequested()
{
    _sim.TryQingLongZhan();
}
void OnEnemyTapped(int enemyId)
{
    _sim.TapFocus(enemyId);
}
```

```
        void OnTowerPlaced(BattleSim.TowerInst tower, int slot)
        {
            _view.SetTower(slot, tower.IsGuanYu);
        }
        void OnTianjiOffered()
        {
            _view.ShowTianjiPanel();
        }
        void OnQingLongFired()
        {
            _view.PlaySlash();
        }
        void OnStateChanged(BattleSim.BattleState state)
        {
            switch (state)
            {
                case BattleSim.BattleState.Prep:
                    _view.HideIntro();
                    break;
                case BattleSim.BattleState.Won:
                    _view.ShowSettle(_sim.Wave, _sim.Kills, _sim.Crystals);
                    break;
                case BattleSim.BattleState.Lost:
                    _view.ShowSettle(-1, _sim.Kills, _sim.Crystals);
                    break;
            }
        }
        sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
        {
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
            {
                var sb = new System.Text.StringBuilder(evtId);
                foreach (var kv in fields)
                {
                    sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
                }
                Debug.Log("[TELEMETRY] " + sb);
            }
        }
    }
}
// BattleSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P03.Newbie
{
    /// &lt;summary&gt;
    ///          C#       Play   Editor headless   NewbieFlowCheck
    ///      06  03   M0      →M1 L1         ×2+  ×1 →M2    +
    /// →M3          →M4    Boss+             →M5    +
```

```
///     A* S                  =
/// &lt;/summary&gt;
public sealed class BattleSim
{
    public enum BattleState { Intro, Prep, Fighting, TianjiPause, Won, Lost }
    public sealed class TowerInst
    {
        public int Id;
        public int Slot;
        public bool IsGuanYu;
        public float Range;
        public float Atk;
        public float Interval;
        public float Cooldown;
    }
    public sealed class EnemyInst
    {
        public int Id;
        public bool Boss;
        public int Seg;
        public float SegT;
        public float X, Y;
        public float Hp, MaxHp;
        public float Speed;
        public bool Dead;
    }
    public BattleState State { get; private set; } = BattleState.Intro;
    readonly List&lt;TowerInst&gt; _towers = new();
    readonly List&lt;EnemyInst&gt; _enemies = new();
    readonly float[] _segLens;
    int _nextId = 1;
    float _stateTimer;
    int _placed;
    int _waveSpawned;
    float _spawnTimer;
    bool _spawning;
    float _waveGapTimer;
    int _lastCleared = -1;
    bool _tianjiDone;
    int _focusId = -1;
    float _focusTimer;
    float _enemySpeedMul = 1f;
    float _atkMul = 1f;
    int _waveBonus;
    public IReadOnlyList&lt;TowerInst&gt; Towers =&gt; _towers;
    public IReadOnlyList&lt;EnemyInst&gt; Enemies =&gt; _enemies;
    public int Wave { get; private set; }
    public int Lives { get; private set; } = NewbieConfig.StartLives;
    public int Crystals { get; private set; }
    public int Kills { get; private set; }
```

```
public int PlacedCount =&gt; _placed;
public float BattleTime { get; private set; }
public event Action&lt;TowerInst, int&gt; TowerPlaced;
public event Action&lt;int&gt; WaveStarted;
public event Action&lt;int&gt; WaveCleared;
public event Action&lt;EnemyInst&gt; EnemySpawned;
public event Action&lt;EnemyInst&gt; EnemyKilled;
public event Action&lt;int&gt; LifeLost;
public event Action TianjiOffered;
public event Action&lt;int&gt; TianjiPicked;
public event Action BossKilled;
public event Action QingLongFired;
public event Action&lt;BattleState&gt; StateChanged;
public BattleSim()
{
    _segLens = new float[NewbieConfig.PathX.Length - 1];
    for (int s = 0; s &lt; _segLens.Length; s++)
    {
        float dx = NewbieConfig.PathX[s + 1] - NewbieConfig.PathX[s];
        float dy = NewbieConfig.PathY[s + 1] - NewbieConfig.PathY[s];
        _segLens[s] = (float)Math.Sqrt(dx * dx + dy * dy);
    }
}
/// &lt;summary&gt;                →  →          &lt;/summary&gt;
public bool PlaceTower(int slot)
{
    if (State != BattleState.Prep) return false;
    if (slot &lt; 0 || slot &gt;= NewbieConfig.SlotX.Length) return false;
    for (int i = 0; i &lt; _towers.Count; i++)
        if (_towers[i].Slot == slot) return false;
    if (_placed &gt;= NewbieConfig.FreeTowerPlacements) return false;
    bool isGuanYu = _placed &lt; NewbieConfig.FreeTowerPlacements - 1;   //
    var def = isGuanYu ? NewbieConfig.GuanYu : NewbieConfig.HuangZhong;
    var tower = new TowerInst
    {
        Id = _nextId++,
        Slot = slot,
        IsGuanYu = isGuanYu,
        Range = def.Range,
        Atk = def.Atk,
        Interval = def.Interval,
    };
    _towers.Add(tower);
    _placed++;
    TowerPlaced?.Invoke(tower, slot);
    return true;
}
/// &lt;summary&gt;                    FocusSeconds  &lt;/summary&gt;
public void TapFocus(int enemyId)
{
```

```
    for (int i = 0; i &lt; _enemies.Count; i++)
        if (_enemies[i].Id == enemyId)
        {
            _focusId = enemyId;
            _focusTimer = NewbieConfig.FocusSeconds;
            return;
        }
}
public bool IsFocused(EnemyInst e) =&gt; e.Id == _focusId &amp;&amp; _focusTimer &gt; 0f;
/// &lt;summary&gt;      ≤40%     →      &lt;/summary&gt;
public bool QingLongZhanReady
{
    get
    {
        for (int i = 0; i &lt; _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (e.Boss &amp;&amp; e.Hp &lt;= e.MaxHp * NewbieConfig.QingLongHpRatio) return true;
        }
        return false;
    }
}
/// &lt;summary&gt;    M4    ·                &lt;/summary&gt;
public bool TryQingLongZhan()
{
    if (State != BattleState.Fighting || !QingLongZhanReady) return false;
    for (int i = 0; i &lt; _enemies.Count; i++)
    {
        var e = _enemies[i];
        if (!e.Boss) continue;
        QingLongFired?.Invoke();
        DamageEnemy(e, e.Hp + 1f);
        return true;
    }
    return false;
}
public void PickTianji(int index)
{
    if (State != BattleState.TianjiPause) return;
    switch (index)
    {
        case 0: _enemySpeedMul = 0.8f; break;
        case 1: _atkMul = 1.25f; break;
        default: _waveBonus = 10; break;
    }
    _waveGapTimer = 0f;
    TianjiPicked?.Invoke(index);
    SetState(BattleState.Fighting);
}
public void Tick(float dt)
```

```
{
    switch (State)
    {
        case BattleState.Intro:
            _stateTimer += dt;
            if (_stateTimer &gt;= NewbieConfig.IntroSeconds) SetState(BattleState.Prep);
            return;
        case BattleState.Prep:
            _stateTimer += dt;
            if (_placed &gt;= 1 &amp;&amp; (_placed &gt;= NewbieConfig.FreeTowerPlacements || _stateTimer &gt;=
            {
                SetState(BattleState.Fighting);
                BeginWave(1);
            }
            return;
        case BattleState.Fighting:
            BattleTime += dt;
            if (_focusTimer &gt; 0f) _focusTimer -= dt;
            TickSpawns(dt);
            TickEnemies(dt);
            TickTowers(dt);
            TickWaves(dt);
            if (Lives &lt;= 0) SetState(BattleState.Lost);
            return;
    }
}
void BeginWave(int w)
{
    Wave = w;
    var (count, _, _) = NewbieConfig.GetWave(w);
    _waveSpawned = 0;
    _spawnTimer = 0f;
    _waveGapTimer = 0f;
    _spawning = true;
    WaveStarted?.Invoke(w);
    if (w == NewbieConfig.NewbieWaves)
    {
        var boss = new EnemyInst
        {
            Id = _nextId++,
            Boss = true,
            Seg = 0,
            SegT = 0f,
            X = NewbieConfig.PathX[0],
            Y = NewbieConfig.PathY[0],
            Hp = NewbieConfig.BossHp,
            MaxHp = NewbieConfig.BossHp,
            Speed = NewbieConfig.BossSpeed,
        };
        _enemies.Add(boss);
```

```
            EnemySpawned?.Invoke(boss);
        }
    }
    void TickSpawns(float dt)
    {
        if (!_spawning) return;
        var (count, hp, speed) = NewbieConfig.GetWave(Wave);
        _spawnTimer += dt;
        while (_spawnTimer &gt;= NewbieConfig.SpawnInterval &amp;&amp; _waveSpawned &lt; count)
        {
            _spawnTimer -= NewbieConfig.SpawnInterval;
            _waveSpawned++;
            var e = new EnemyInst
            {
                Id = _nextId++,
                Seg = 0,
                SegT = 0f,
                X = NewbieConfig.PathX[0],
                Y = NewbieConfig.PathY[0],
                Hp = hp,
                MaxHp = hp,
                Speed = speed,
            };
            _enemies.Add(e);
            EnemySpawned?.Invoke(e);
        }
        if (_waveSpawned &gt;= count) _spawning = false;
    }
    void TickEnemies(float dt)
    {
        for (int i = _enemies.Count - 1; i &gt;= 0; i--)
        {
            var e = _enemies[i];
            float move = e.Speed * _enemySpeedMul * dt;
            while (move &gt; 0f)
            {
                float segLen = _segLens[e.Seg];
                float remain = (1f - e.SegT) * segLen;
                if (move &lt; remain)
                {
                    e.SegT += move / segLen;
                    move = 0f;
                }
                else
                {
                    move -= remain;
                    e.Seg++;
                    e.SegT = 0f;
                    if (e.Seg &gt;= _segLens.Length)
                    {
```

```
                    if (e.Boss)
                    {
                        //     =
                        e.Seg = 0;
                        e.SegT = 0f;
                        Lives = Math.Max(0, Lives - 1);
                        LifeLost?.Invoke(Lives);
                    }
                    else
                    {
                        _enemies.RemoveAt(i);
                        Lives = Math.Max(0, Lives - 1);
                        LifeLost?.Invoke(Lives);
                    }
                    break;
                }
            }
        }
        if (e.Seg &lt; _segLens.Length)
        {
            e.X = Lerp(NewbieConfig.PathX[e.Seg], NewbieConfig.PathX[e.Seg + 1], e.SegT);
            e.Y = Lerp(NewbieConfig.PathY[e.Seg], NewbieConfig.PathY[e.Seg + 1], e.SegT);
        }
    }
}
static float Lerp(float a, float b, float t) =&gt; a + (b - a) * t;
void TickTowers(float dt)
{
    for (int i = 0; i &lt; _towers.Count; i++)
    {
        var t = _towers[i];
        t.Cooldown -= dt;
        if (t.Cooldown &gt; 0f) continue;
        bool any = false;
        if (t.IsGuanYu)
        {
            //
            float r2 = t.Range * t.Range;
            for (int j = _enemies.Count - 1; j &gt;= 0; j--)
            {
                var e = _enemies[j];
                float dx = e.X - NewbieConfig.SlotX[t.Slot], dy = e.Y - NewbieConfig.SlotY[t.Slot];
                if (dx * dx + dy * dy &gt; r2) continue;
                any = true;
                DamageEnemy(e, t.Atk * _atkMul);
            }
        }
        else
        {
            //
```

```
                float r2 = t.Range * t.Range;
                EnemyInst target = null;
                float best = float.MaxValue;
                for (int j = 0; j &lt; _enemies.Count; j++)
                {
                    var e = _enemies[j];
                    float dx = e.X - NewbieConfig.SlotX[t.Slot], dy = e.Y - NewbieConfig.SlotY[t.Slot];
                    float d2 = dx * dx + dy * dy;
                    if (d2 &gt; r2) continue;
                    if (IsFocused(e)) { target = e; best = -1f; break; }
                    if (d2 &lt; best) { best = d2; target = e; }
                }
                if (target != null)
                {
                    any = true;
                    DamageEnemy(target, t.Atk * _atkMul);
                }
            }
            t.Cooldown = any ? t.Interval : 0.05f;
        }
    }
    void DamageEnemy(EnemyInst e, float dmg)
    {
        e.Hp -= dmg;
        if (e.Hp &gt; 0f) return;
        e.Dead = true;
        _enemies.Remove(e);
        Kills++;
        Crystals += NewbieConfig.KillCrystal;
        if (e.Boss)
        {
            //
            for (int i = _enemies.Count - 1; i &gt;= 0; i--)
            {
                var rest = _enemies[i];
                rest.Dead = true;
                _enemies.RemoveAt(i);
                Kills++;
                Crystals += NewbieConfig.KillCrystal;
                EnemyKilled?.Invoke(rest);
            }
            BossKilled?.Invoke();
            EnemyKilled?.Invoke(e);
            SetState(BattleState.Won);
            return;
        }
        EnemyKilled?.Invoke(e);
    }
    void TickWaves(float dt)
    {
```

```
            if (_spawning || _enemies.Count &gt; 0) return;
            _waveGapTimer += dt;
            if (_waveGapTimer &lt; NewbieConfig.WaveGapSeconds) return;
            int cleared = Wave;
            if (cleared &gt;= NewbieConfig.NewbieWaves) return;   //    Boss
            if (_lastCleared != cleared)
            {
                _lastCleared = cleared;
                Crystals += NewbieConfig.WaveClearCrystal + _waveBonus;
                WaveCleared?.Invoke(cleared);
            }
            if (cleared == 2 &amp;&amp; !_tianjiDone)
            {
                _tianjiDone = true;
                SetState(BattleState.TianjiPause);
                TianjiOffered?.Invoke();
                return;
            }
            BeginWave(cleared + 1);
        }
        void SetState(BattleState s)
        {
            if (State == s) return;
            State = s;
            _stateTimer = 0f;
            StateChanged?.Invoke(s);
        }
    }
}
// BattleView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P03.Newbie
{
    /// &lt;summary&gt;
    ///            UGUI——   ×   ×  ×
    ///   UI Initialize          YAML      =      /     /        /
    ///
    /// &lt;/summary&gt;
    public sealed class BattleView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        readonly Image[] _slotImages = new Image[NewbieConfig.SlotX.Length];
        readonly Text[] _slotLabels = new Text[NewbieConfig.SlotX.Length];
        Text _waveText, _livesText, _crystalText, _killsText, _hintText;
        Button _qingLongButton;
```

```
Image _qingLongImage;
RectTransform _introOverlay, _tianjiPanel, _slashOverlay, _settlePanel;
Text _settleStats;
readonly Dictionary&lt;int, EnemyVisual&gt; _enemies = new();
readonly List&lt;int&gt; _scratch = new();
public event Action&lt;int&gt; SlotTapped;
public event Action&lt;int&gt; EnemyTapped;
public event Action&lt;int&gt; TianjiPicked;
public event Action QingLongRequested;
static readonly Color RevRed = new Color(0.757f, 0.153f, 0.176f);      //     #C1272D
static readonly Color PaperYellow = new Color(0.929f, 0.871f, 0.706f);  //
static readonly Color InkBlack = new Color(0.043f, 0.043f, 0.047f);     //
static readonly Color DoomGreen = new Color(0.298f, 0.686f, 0.314f);    //     #4CAF50
static readonly Color PathCol = new Color(0.16f, 0.08f, 0.08f);
public void Initialize(Canvas canvas, BattleSim sim)
{
    _canvas = canvas;
    _square = MakeSquareSprite();
    BuildBackdrop();
    BuildHud();
    BuildPath();
    BuildSlots();
    BuildIntroOverlay();
    BuildTianjiPanel();
    BuildSlashOverlay();
    BuildSettlePanel();
    SetHint("    ……");
}
// ————    ————
/// &lt;summary&gt;X027 generated art: fullscreen backdrop behind all UI; skipped
/// entirely when the art key is missing so headless checks stay green.&lt;/summary&gt;
void BuildBackdrop()
{
    var art = Sy.Core.ArtRuntime.Get("P03_scene_01");
    if (art == null) return;
    var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    rt.SetAsFirstSibling();
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
    image.raycastTarget = false;
}
void BuildHud()
{
```

```
    var root = _canvas.transform;
    _waveText = MakeText(root, "L1    0/4", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 380, 70);
    _waveText.color = PaperYellow;
    _livesText = MakeText(root, "   5", 46, new Vector2(0, 880), TextAnchor.UpperRight, 340, 70);
    _crystalText = MakeText(root, "   0", 44, new Vector2(0, 820), TextAnchor.UpperLeft, 380, 60);
    _crystalText.color = PaperYellow;
    _killsText = MakeText(root, "   0", 44, new Vector2(0, 820), TextAnchor.UpperRight, 380, 60);
    _hintText = MakeText(root, "", 40, new Vector2(0, -780), TextAnchor.MiddleCenter, 1020, 150);
    _hintText.color = new Color(0.95f, 0.87f, 0.6f);
    var btnGo = new GameObject("QingLongButton", typeof(Image), typeof(Button));
    btnGo.transform.SetParent(root, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
    btnRt.anchoredPosition = new Vector2(0, 120);
    btnRt.sizeDelta = new Vector2(460, 140);
    _qingLongImage = btnGo.GetComponent&lt;Image&gt;();
    _qingLongImage.sprite = _square;
    _qingLongImage.color = new Color(0.35f, 0.35f, 0.35f);
    _qingLongButton = btnGo.GetComponent&lt;Button&gt;();
    _qingLongButton.interactable = false;
    MakeText(btnGo.transform, "    ", 54, Vector2.zero, TextAnchor.MiddleCenter, 460, 140).color = InkBlack;
    _qingLongButton.onClick.AddListener(() =&gt; QingLongRequested?.Invoke());
}
void BuildPath()
{
    for (int s = 0; s &lt; NewbieConfig.PathX.Length - 1; s++)
    {
        float x0 = NewbieConfig.PathX[s], y0 = NewbieConfig.PathY[s];
        float x1 = NewbieConfig.PathX[s + 1], y1 = NewbieConfig.PathY[s + 1];
        bool horiz = Math.Abs(y1 - y0) &lt; 1f;
        var go = new GameObject("PathSeg_" + s, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2((x0 + x1) * 0.5f, (y0 + y1) * 0.5f);
        rt.sizeDelta = horiz
            ? new Vector2(Math.Abs(x1 - x0) + 90f, 90f)
            : new Vector2(90f, Math.Abs(y1 - y0) + 90f);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = PathCol;
    }
    MakeText(_canvas.transform, "    ", 34, new Vector2(-460f, 820f), TextAnchor.MiddleCenter, 260, 50).color =
    MakeText(_canvas.transform, "  ", 34, new Vector2(-500f, -660f), TextAnchor.MiddleCenter, 260, 50).color = P
}
void BuildSlots()
{
    for (int i = 0; i &lt; NewbieConfig.SlotX.Length; i++)
    {
        int index = i;
```

```
            var go = new GameObject("Slot_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
            go.transform.SetParent(_canvas.transform, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(NewbieConfig.SlotX[i], NewbieConfig.SlotY[i]);
            rt.sizeDelta = new Vector2(140, 140);
            var img = go.GetComponent&lt;Image&gt;();
            var slotArt = Sy.Core.ArtRuntime.Get("P03_unit_01");
            if (slotArt != null)
            {
                img.sprite = slotArt;
                img.color = Color.white;
                img.preserveAspect = true;
            }
            else
            {
                img.sprite = _square;
                img.color = new Color(0.13f, 0.13f, 0.13f, 0.9f);
            }
            _slotImages[i] = img;
            _slotLabels[i] = MakeText(go.transform, "  ", 34, Vector2.zero, TextAnchor.MiddleCenter, 140, 140);
            _slotLabels[i].color = new Color(0.55f, 0.52f, 0.45f);
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; SlotTapped?.Invoke(index));
        }
    }
    void BuildIntroOverlay()
    {
        _introOverlay = MakeOverlayPanel("IntroOverlay", new Color(0.06f, 0.02f, 0.02f, 0.95f));
        MakeText(_introOverlay, "       ", 100, new Vector2(0, 100), TextAnchor.MiddleCenter, 1000, 130).color = Rev
        MakeText(_introOverlay, "     · L1    ", 46, new Vector2(0, -60), TextAnchor.MiddleCenter, 1000, 80).color =
    }
    void BuildTianjiPanel()
    {
        _tianjiPanel = MakeOverlayPanel("TianjiPanel", new Color(0.03f, 0.03f, 0.03f, 0.92f));
        MakeText(_tianjiPanel, "   ·    ", 76, new Vector2(0, 560), TextAnchor.MiddleCenter, 1000, 100).color = Pape
        for (int i = 0; i &lt; NewbieConfig.TianjiCards.Length; i++)
        {
            int index = i;
            var card = NewbieConfig.TianjiCards[i];
            var go = new GameObject("Tianji_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
            go.transform.SetParent(_tianjiPanel, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(-330 + i * 330, 0);
            rt.sizeDelta = new Vector2(300, 420);
            var img = go.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = i == 0 ? new Color(0.55f, 0.12f, 0.12f) : new Color(0.14f, 0.16f, 0.14f);   //
            var label = MakeText(go.transform, " " + card.Name + " ", 56, new Vector2(0, 120), TextAnchor.MiddleCent
            label.color = PaperYellow;
```

```
            MakeText(go.transform, card.Desc, 38, new Vector2(0, -40), TextAnchor.MiddleCenter, 280, 200).color = ne
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt;
            {
                _tianjiPanel.gameObject.SetActive(false);
                TianjiPicked?.Invoke(index);
            });
        }
        MakeText(_tianjiPanel, "         ——    ", 40, new Vector2(0, -400), TextAnchor.MiddleCenter, 1000, 80).color
        _tianjiPanel.gameObject.SetActive(false);
    }
    void BuildSlashOverlay()
    {
        _slashOverlay = MakeOverlayPanel("SlashOverlay", new Color(1f, 0.98f, 0.92f, 0.75f));
        var lineGo = new GameObject("SlashLine", typeof(RectTransform), typeof(Image));
        lineGo.transform.SetParent(_slashOverlay, false);
        var lineRt = (RectTransform)lineGo.transform;
        lineRt.anchorMin = lineRt.anchorMax = new Vector2(0.5f, 0.5f);
        lineRt.anchoredPosition = Vector2.zero;
        lineRt.sizeDelta = new Vector2(2200, 16);
        lineRt.localRotation = Quaternion.Euler(0f, 0f, 24f);
        var lineImg = lineGo.GetComponent&lt;Image&gt;();
        lineImg.sprite = _square;
        lineImg.color = RevRed;
        MakeText(_slashOverlay, "      ", 110, new Vector2(0, -180), TextAnchor.MiddleCenter, 1000, 140).color = Ink
        _slashOverlay.gameObject.SetActive(false);
    }
    void BuildSettlePanel()
    {
        _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.03f, 0.03f, 0.03f, 0.96f));
        MakeText(_settlePanel, "L1     ", 90, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 110).color = Paper
        _settleStats = MakeText(_settlePanel, "", 46, new Vector2(0, 380), TextAnchor.MiddleCenter, 1000, 260);
        MakeActionButton(_settlePanel, "    ", new Vector2(-260, -260), () =&gt; UnityEngine.SceneManagement.SceneMa
        MakeActionButton(_settlePanel, "     ", new Vector2(260, -260), () =&gt; UnityEngine.SceneManagement.SceneMa
        _settlePanel.gameObject.SetActive(false);
    }
    // ——— reconcile ———
    public void SyncView(BattleSim sim)
    {
        ReconcileEnemies(sim);
        foreach (var kv in _enemies)
        {
            var v = kv.Value;
            v.Rt.anchoredPosition = new Vector2(v.X, v.Y);
            if (v.Fill != null) v.Fill.fillAmount = v.MaxHp &gt; 0f ? v.Hp / v.MaxHp : 0f;
            bool focused = v.Focused;
            if (focused != v.LastFocused)
            {
                v.LastFocused = focused;
                v.FocusLabel.gameObject.SetActive(focused);
            }
```

```
        }
        bool ready = sim.QingLongZhanReady;
        if (_qingLongButton.interactable != ready)
        {
            _qingLongButton.interactable = ready;
            _qingLongImage.color = ready ? RevRed : new Color(0.35f, 0.35f, 0.35f);
        }
    }
    void ReconcileEnemies(BattleSim sim)
    {
        _scratch.Clear();
        foreach (var kv in _enemies)
            if (!ContainsEnemy(sim, kv.Key)) _scratch.Add(kv.Key);
        for (int i = 0; i &lt; _scratch.Count; i++)
        {
            Destroy(_enemies[_scratch[i]].Go);
            _enemies.Remove(_scratch[i]);
        }
        for (int i = 0; i &lt; sim.Enemies.Count; i++)
        {
            var e = sim.Enemies[i];
            if (!_enemies.TryGetValue(e.Id, out var v))
            {
                v = CreateEnemyVisual(e);
                _enemies[e.Id] = v;
            }
            v.X = e.X;
            v.Y = e.Y;
            v.Hp = e.Hp;
            v.MaxHp = e.MaxHp;
            v.Focused = sim.IsFocused(e);
        }
    }
    static bool ContainsEnemy(BattleSim sim, int id)
    {
        for (int i = 0; i &lt; sim.Enemies.Count; i++)
            if (sim.Enemies[i].Id == id) return true;
        return false;
    }
    EnemyVisual CreateEnemyVisual(BattleSim.EnemyInst e)
    {
        int id = e.Id;
        var go = new GameObject("Enemy_" + e.Id, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.sizeDelta = e.Boss ? new Vector2(160, 160) : new Vector2(70, 70);
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = e.Boss ? PaperYellow : new Color(0.35f, 0.42f, 0.33f);
```

```
    if (e.Boss)
        MakeText(go.transform, " ", 64, Vector2.zero, TextAnchor.MiddleCenter, 160, 160).color = InkBlack;
    Image fill = null;
    var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
    hpGo.transform.SetParent(go.transform, false);
    var hpRt = (RectTransform)hpGo.transform;
    hpRt.anchorMin = new Vector2(0f, 1f);
    hpRt.anchorMax = new Vector2(1f, 1f);
    hpRt.anchoredPosition = new Vector2(0, -6);
    hpRt.sizeDelta = new Vector2(-8, 10);
    fill = hpGo.GetComponent&lt;Image&gt;();
    fill.sprite = _square;
    fill.type = Image.Type.Filled;
    fill.fillMethod = Image.FillMethod.Horizontal;
    fill.color = new Color(0.2f, 0.9f, 0.3f);
    var focusLabel = MakeText(go.transform, "  ", 34, new Vector2(0, 120), TextAnchor.MiddleCenter, 220, 50);
    focusLabel.color = PaperYellow;
    focusLabel.gameObject.SetActive(false);
    go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; EnemyTapped?.Invoke(id));
    return new EnemyVisual
    {
        Go = go,
        Rt = rt,
        Fill = fill,
        FocusLabel = focusLabel.gameObject,
        X = e.X,
        Y = e.Y,
        Hp = e.Hp,
        MaxHp = e.MaxHp,
    };
}
// ——— HUD    ———
public void SetWave(int w) =&gt; _waveText.text = "L1    " + w + "/" + NewbieConfig.NewbieWaves;
public void SetLives(int l) =&gt; _livesText.text = "   " + l;
public void SetCrystals(int c) =&gt; _crystalText.text = "   " + c;
public void SetKills(int k) =&gt; _killsText.text = "   " + k;
public void SetHint(string hint) =&gt; _hintText.text = hint;
public void HideIntro() =&gt; _introOverlay.gameObject.SetActive(false);
public void ShowTianjiPanel() =&gt; _tianjiPanel.gameObject.SetActive(true);
public void SetTower(int slot, bool isGuanYu)
{
    _slotImages[slot].color = isGuanYu ? RevRed : DoomGreen;
    _slotLabels[slot].text = isGuanYu ? " " : " ";
    _slotLabels[slot].color = InkBlack;
    _slotLabels[slot].fontSize = 64;
}
public void PlaySlash()
{
    StartCoroutine(SlashRoutine());
}
```

```
IEnumerator SlashRoutine()
{
    _slashOverlay.gameObject.SetActive(true);
    yield return new WaitForSeconds(0.5f);   //       06  M4
    _slashOverlay.gameObject.SetActive(false);
}
public void ShowSettle(int wave, int kills, int crystals)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = wave &gt;= 0
        ? $"   {wave}/{NewbieConfig.NewbieWaves} ·    {kills} ·    {crystals}\n&lt;size=40&gt;    L1-DEMO-0001
        : $"  ……   {kills} ·    {crystals}\n&lt;size=40&gt;      ——    &lt;/size&gt;";
}
// ————    ————
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    return rt;
}
void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
{
    var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(420, 120);
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = DoomGreen;
    MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
    go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
}
static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent&lt;Text&gt;();
```

```
            t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            t.supportRichText = true;
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
        // ─────        ─────
        sealed class EnemyVisual
        {
            public GameObject Go;
            public RectTransform Rt;
            public Image Fill;
            public GameObject FocusLabel;
            public bool LastFocused;
            public bool Focused;
            public float X, Y;
            public float Hp, MaxHp;
        }
    }
}
// NewbieConfig.cs
namespace Sy.P03.Newbie
{
    public readonly struct TowerDef
    {
        public readonly string Name;
        public readonly float Range;
        public readonly float Atk;
        public readonly float Interval;
        public readonly bool Aoe;   //    =
        public TowerDef(string name, float range, float atk, float interval, bool aoe)
        {
            Name = name;
            Range = range;
            Atk = atk;
            Interval = interval;
            Aoe = aoe;
        }
    }
```

```
    /// &lt;summary&gt;
    /// P03         ·           V0.1——            X009+IConfigService      Design/configs/P03 JSON
    ///    06    03 M0-M5     /L1  /     /      /
    ///       #C1272D ×     ×    ×    #4CAF50  A* S     ——   AI
    /// &lt;/summary&gt;
    public static class NewbieConfig
    {
        public const string GameTitle = "     ";
        public const string GameSlogan = "     ·     ";
        public const int NewbieWaves = 4;              // L1   4  W4=    Boss
        public const int StartLives = 5;                //      -1         /
        public const float IntroSeconds = 3f;          // M0
        public const float PrepStartSeconds = 15f;     //         1
        public const int FreeTowerPlacements = 3;       //        ×2+  ×1
        public const float SpawnInterval = 0.8f;
        public const float WaveGapSeconds = 2f;
        public const float MaxTutorialSeconds = 210f;  //      ≤3.5min 06  ②
        public const int KillCrystal = 2;               //        +2/  +8
        public const int WaveClearCrystal = 8;
        /// &lt;summary&gt;        /      &lt;/summary&gt;
        public static readonly TowerDef GuanYu = new("   ", range: 280f, atk: 12f, interval: 0.8f, aoe: true);
        public static readonly TowerDef HuangZhong = new("   ", range: 520f, atk: 26f, interval: 1.1f, aoe: false);
        /// &lt;summary&gt;S      8 7    →     &lt;/summary&gt;
        public static readonly float[] PathX = { -460f, 340f, 340f, -340f, -340f, 340f, 340f, -500f };
        public static readonly float[] PathY = { 740f, 740f, 300f, 300f, -140f, -140f, -580f, -580f };
        /// &lt;summary&gt;8    3 =        /  /            &lt;/summary&gt;
        public static readonly float[] SlotX = { 0f, 0f, 0f, -320f, 320f, -320f, 320f, -420f };
        public static readonly float[] SlotY = { 620f, 420f, -280f, 620f, 420f, -280f, 620f, -280f };
        public const float BossHp = 900f;
        public const float BossSpeed = 55f;
        public const float QingLongHpRatio = 0.4f;      //       ≤40%→
        public const float FocusSeconds = 4f;            //
        /// &lt;summary&gt;           L1   W4=4  +      &lt;/summary&gt;
        public static (int Count, float Hp, float Speed) GetWave(int w) =&gt; w switch
        {
            1 =&gt; (5, 26f, 75f),
            2 =&gt; (7, 32f, 78f),
            3 =&gt; (10, 38f, 82f),
            _ =&gt; (4, 40f, 86f),
        };
        /// &lt;summary&gt;      06  M3         =       &lt;/summary&gt;
        public static readonly (string Name, string Desc)[] TianjiCards =
        {
            ("   ", "     -20%     "),
            ("  ", "   +25%     "),
            ("  ", "      +10"),
        };
    }
}
// TutorialFlow.cs
```

```
using System;
namespace Sy.P03.Newbie
{
    /// &lt;summary&gt;
    ///        evt_newbie_step 1-5 ①   ②    ③   ④ Boss⑤   +
    ///      BattleSim    /    /             /  /    ——
    /// Play   headless   NewbieFlowCheck
    /// &lt;/summary&gt;
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action&lt;string&gt; _hint;
        readonly Action&lt;int&gt; _onStep;
        bool _firstTowerSeen;
        public TutorialFlow(BattleSim sim, Action&lt;string&gt; hint, Action&lt;int&gt; onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.TowerPlaced += OnTowerPlaced;
            sim.WaveCleared += OnWaveCleared;
            sim.TianjiOffered += OnTianjiOffered;
            sim.TianjiPicked += OnTianjiPicked;
            sim.BossKilled += OnBossKilled;
            sim.StateChanged += OnStateChanged;
            _hint("    ——L1             ×2 +   ×1    ");
        }
        void Advance(int step)
        {
            if (Step &gt;= step) return;
            Step = step;
            _onStep(step);
        }
        void OnTowerPlaced(BattleSim.TowerInst tower, int slot)
        {
            if (!_firstTowerSeen)
            {
                _firstTowerSeen = true;
                Advance(1);
                _hint("              ——            ");
                return;
            }
            _hint(tower.IsGuanYu ? "     ——    " : "     ——         ");
        }
        void OnWaveCleared(int wave)
        {
            if (wave == 1)
            {
                Advance(2);
                _hint("              ——        ");
            }
```

```
        }
        void OnTianjiOffered()
        {
            _hint("     ——                -20%");
        }
        void OnTianjiPicked(int index)
        {
            Advance(3);
            _hint(" " + NewbieConfig.TianjiCards[index].Name + "    " + NewbieConfig.TianjiCards[index].Desc + "——    ")
        }
        void OnBossKilled()
        {
            Advance(4);
            _hint("           ——L1    ");
        }
        void OnStateChanged(BattleSim.BattleState state)
        {
            if (state == BattleSim.BattleState.Won)
            {
                Advance(5);
                _hint("L1           ——         ");
            }
        }
    }
}
```